

4D 成像毫米波雷达 · 树莓派使用说明

Raspberry Pi 4B / Raspberry Pi OS · radarpi 1.0.0

生成日期：2026-09-16

本说明配合厂家《4D 成像毫米波雷达使用说明》使用：雷达的安装、供电、接线与安全要求仍以原手册为准。本说明覆盖树莓派上位机 radarpi 的安装、操作、配置、录制回放与故障排除。

原 Windows 上位机的固件升级（Upgrade）与校准向导（Calibration）未移植，仍需 Windows 电脑操作；其余功能在树莓派上等价可用。

目录

- 一、系统组成与工作原理
- 二、安全注意事项
- 三、硬件准备与接线
- 四、安装
- 五、快速开始（5 分钟跑通）
- 六、Web 上位机界面说明
- 七、pygame 桌面版上位机
- 八、雷达参数配置
- 九、数据录制与回放
- 十、命令行参考
- 十一、开机自启与无人值守
- 十二、故障排除
- 十三、日常维护
- 十四、技术指标（雷达）
- 十五、ndarray 矩阵接口（算法接入）
- 附录 A：CH343/CH344 串口小板驱动
- 附录 B：数据协议说明
- 附录 C：与原 Windows 上位机的差异

本 PDF 由 docs/build_manual_pdf.py 从 docs/使用说明.md 自动生成，改文档请改 Markdown 源文件。

4D 成像毫米波雷达 · 树莓派 (Raspberry Pi OS) 使用说明

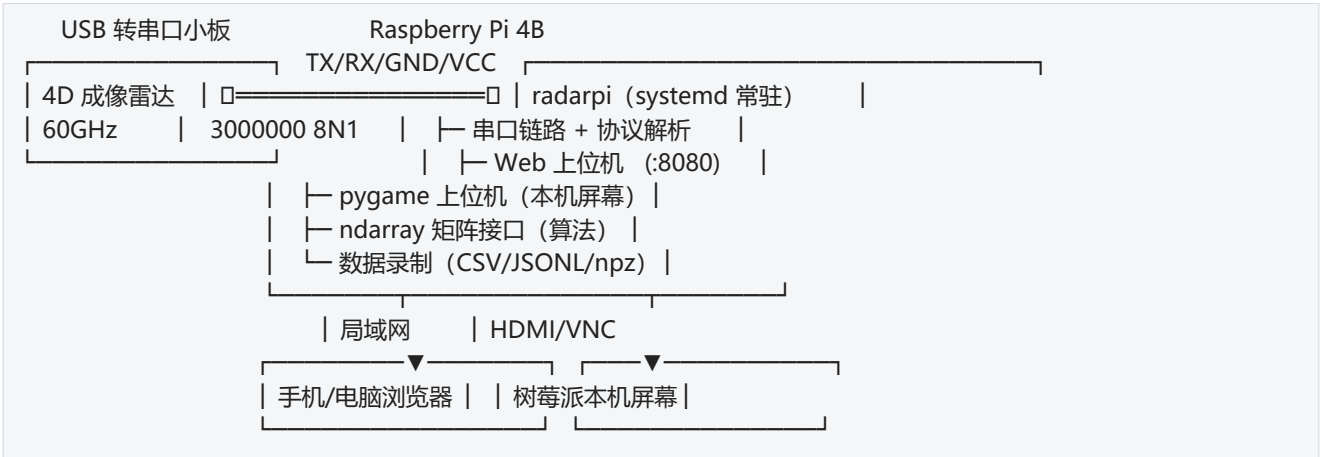
适用于 Raspberry Pi 4B + Raspberry Pi OS (Bookworm / Trixie, 64 位或 32 位) , 配套软件 radarpi 1.0.0。

本说明与厂家《4D 成像毫米波雷达使用说明》配套使用：**雷达本身的安装、供电、接线、安全注意事项仍以原手册为准**，本说明只覆盖"把上位机换到树莓派上"之后的操作。

目录

- 系统组成与工作原理
- 安全注意事项
- 硬件准备与接线
- 安装
- 快速开始 (5 分钟跑通)
- Web 上位机界面说明
- pygame 桌面版上位机
- 雷达参数配置
- 数据录制与回放
- 命令行参考
- 开机自启与无人值守
- 故障排除
- 日常维护
- 技术指标 (雷达)
- ndarray 矩阵接口 (算法接入)
- 附录 A: CH343/CH344 串口小板驱动
- 附录 B: 数据协议说明
- 附录 C: 与原 Windows 上位机的差异

一、系统组成与工作原理



树莓派承担原 Windows 电脑的角色：给雷达下发配置指令、接收并解析点云、把结果实时显示出来并保存。界面有两种，功能相同、按场合选：

- **Web 版** (radarpi serve)：浏览器打开 `http://<树莓派IP>:8080/` 即可看，手机、电脑都能看，适合远程与长期监视（第六章）；
 - **pygame 版** (radarpi view)：原生窗口程序，适合树莓派接了显示器或用 VNC 的场合，延迟更低，并能把算法处理结果直接画出来（第七章）。
- 此外还提供 **ndarray 矩阵接口**（第十五章）：把点云转成 numpy 矩阵（含 BEV、体素），方便做聚类、跟踪、建图或喂给神经网络。

二、安全注意事项

以下条目沿用原手册要求，务必遵守：

- **安装环境**：远离高压线、大型电机等强电磁干扰源；探测面不要被金属遮挡；远离热源与易积水处。
- **电气安全**：确认供电电压与额定值匹配（**5-12V DC，供电能力 1A 以上**，否则雷达无法启动），正负极严禁接反；接线、拆线前先断电。
- **使用防护**：避免在雷达正前方 0.5 米内长时间停留；禁止拆解雷达外壳。
- **本移植版的额外提醒**：
 - 树莓派请用**官方电源适配器（5V/3A）**，USB 口同时给雷达供电时容易触发欠压（现象：树莓派重启、dmesg 报 Under-voltage detected）。建议雷达用独立电源，串口小板只接 TX/RX/GND。
 - 树莓派长期运行时加装散热片或风扇，避免降频；`vcgencmd measure_temp` 查看温度。

三、硬件准备与接线

3.1 物料清单

物料	要求	备注
树莓派 4B	2GB 内存以上	1GB 版本也能跑，但浏览器推送点数要调小
TF 卡	16GB 以上，Class 10	建议 32GB，录制点云比较占空间
USB 转串口小板	必须支持 3000000 波特率	首选 FT232 （内核自带驱动）；CH343/CH344 见附录 A
雷达电源	5-12V DC，≥1A	不要用串口小板的 3.3V
杜邦线	TX / RX / GND（+VCC 视供电方式）	TX 与 RX 必须交叉

不要用 CH340 / CH341 小板：最高只有 2 Mbit/s，达不到手册要求的 3000000。

3.2 接线（雷达为裸板时，按丝印）

雷达	串口小板	说明
RXD	TXD	交叉连接
TXD	RXD	交叉连接
GND	GND	必须共地
VCC	5V（仅在确认能提供 1A 时）	建议雷达用独立电源，此脚不接

SEL、CAN_H、CAN_L、PWM 本移植版不使用，可悬空；PWM 是雷达正常工作时的脉冲输出，可用于示波器判断雷达是否在跑。

3.3 接线（带外壳雷达，按线束颜色，摘自原手册）

顺序	颜色	定义	说明
1	红色	VCC	5-12V 稳定供电，1A 以上载流能力
2	黑色	RXD	3.3V UART
3	黄色	TXD	3.3V UART
4	白色	SEL	NC
5	绿色	CAN_H	CAN 接口（本移植版不用）
6	橙色	CAN_L	CAN 接口（本移植版不用）
7	蓝色	PWM	PWM 脉冲输出，雷达正常工作时输出
8	棕色	GND	地

3.4 上电检查

接通电源后观察雷达指示灯：**电源灯常亮绿色、通信灯闪烁绿色**表示正常。树莓派侧执行：

```
lsusb          # 应能看到 0403:6001 (FTDI) 或 1a86:xxxx (WCH)
ls /dev/ttyUSB* /dev/ttyACM* /dev/radar # 应出现设备节点
```

四、安装

4.1 准备系统

```
sudo apt update && sudo apt full-upgrade -y # 建议先更新
sudo raspi-config # 可选：开启 SSH / 配置 Wi-Fi，方便远程操作
```

4.2 安装 radarpi

方式一：.deb 安装包（推荐）

```
# 把 radarpi_1.0.0_all.deb 拷到树莓派（U 盘、scp、sftp 均可）
sudo apt install ./radarpi_1.0.0_all.deb
```

方式二：源码安装

```
sudo ./packaging/install.sh
```

两种方式效果相同，都会：

- 创建专用系统用户 radarpi（服务以该用户运行，权限最小）；

- 把当前登录用户加入 dialout 组 —— **需要重新登录（或重启）后生效**；
- 写入 udev 规则：串口权限、屏蔽 ModemManager/brltty 抢占、给雷达一个固定别名

/dev/radar；

- 注册并启动 radarpi-web 服务，开机自启。

安装结束会打印网页地址与常用命令。

安装包**不需要联网**（不装任何 Python 三方包），也不依赖 pip。Web 版装完即可用；**pygame 桌面版与 ndarray 矩阵接口需要额外的两个 apt 包**，按需安装：`bash sudo apt install -y python3-pygame python3-numpy fonts- noto-cjk` 不装也不影响 Web 版运行（radarpi view` 会提示缺什么）。

4.3 验证安装

```
radarpi version
radarpi doctor      # 体检：串口驱动、权限、波特率、抢占情况
radarpi doctor --probe # 真机联调：打开串口读 3 秒，看能否解出帧
systemctl status radarpi-web
```

radarpi doctor --probe 输出里出现 收到 N 字节，解析出 M 帧 / K 点 就说明整条链路通了。

五、快速开始（5 分钟跑通）

```
# 1) 接线、上电，确认雷达电源灯常亮、通信灯闪烁
# 2) 体检 + 联调
radarpi doctor --probe

# 3) 启动 Web 上位机（安装包已自动启动，可跳过）
radarpi serve
# 或者：启动 pygame 桌面版（树莓派接了显示器 / 用 VNC 时）
# 需要先装依赖：sudo apt install -y python3-pygame python3-numpy fonts- noto-cjk
# sudo systemctl stop radarpi-web    # 串口独占，先把 Web 服务停掉
# radarpi view

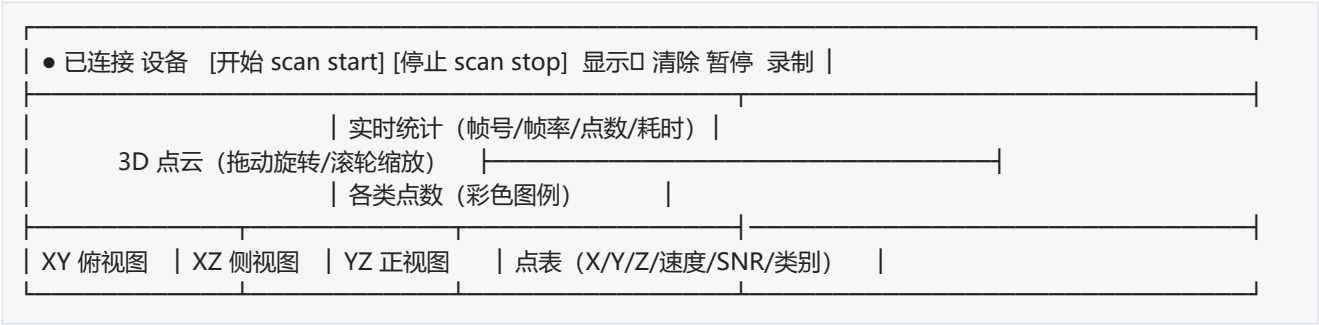
# 4) 浏览器打开（树莓派 IP 用 hostname -I 查）
# http://192.168.x.x:8080/
```

页面打开后应当看到：右上角“已连接”，3D 点云面板里出现彩色点，底部状态栏显示帧号与帧率。若雷达没数据，先在网页“设置 → 雷达指令”里点一下 **开始发送点云（scan start）** 的“下发”。

手上还没有雷达？ 先用内置仿真熟悉界面：

```
radarpi serve --simulate    # Web 版仿真一个 6m×5m 房间 + 走动的人
radarpi view --simulate    # pygame 版同样支持
```

六、Web 上位机界面说明



6.1 工具栏

控件	作用
• 状态灯	绿色=链路正常；黄色=连接着但超过 2.5 秒没收到帧；红色=未连接
开始 scan start / 停止 scan stop	让雷达开始/停止输出点云（对应手册的 scan start / scan stop）
显示	全部点 / 仅动态点 / 仅微动点 / 仅航迹，用于排除干扰观察目标
清除	清空当前画面（不影响后台接收）
暂停	冻结画面刷新，数据仍在后台接收与录制（截图、讲解时用）
开始录制 / 停止录制	把点云写入 CSV 或 JSONL 文件，按钮旁显示文件名
设置	打开右侧抽屉：雷达指令下发、配置文件编辑、自定义指令
状态 JSON	查看原始状态（排查问题、做二次开发时用）

6.2 视图

- **3D 点云**：鼠标左键拖动旋转、滚轮缩放。X 轴红、Y 轴绿、Z 轴蓝，原点即雷达位置；地面有 1 米网格。
- **XY 俯视图**：最常见的工作视图，向上是正前方 Y。
- **XZ 侧视图 / YZ 正视图**：用来判断目标高度与前后位置。
- **点表**：列出本帧每个点的坐标、速度、SNR 与类别，可取消"跟随最新帧"后停止滚动，方便逐个核对。

6.3 点的颜色（6 类，与手册一一对应）

颜色	类别	含义
亮黄	动态高置信度点	运动目标，最可靠
暗黄	动态低置信度点	静止/弱反射（墙面、地面多属此类）
青色	长时微动高置信度点	长时间存在的小幅运动（呼吸、风扇）
暗青	长时微动低置信度点	同上，置信度低
绿色	短时微动高置信度点	短时间小幅运动
暗绿	短时微动低置信度点	同上，置信度低
白色圆圈	航迹	雷达内部跟踪器的输出（若有）

6.4 实时统计

帧号、帧率、本帧点数、数据率（串口字节数/秒）、帧周期、帧间隔、BB 处理时间、Post-BB 处理时间、结果传输时间、网页客户端数。**数据率**能直观反映串口是否正常：3000000 波特率下点云密集时通常几十 KB/s 到几百 KB/s。

页面底部出现红色"链路告警：重同步 N 次..."说明串口数据在丢字节。优先检查 波特率是否真的是 3000000、串口小板是否支持、USB 线是否过长或劣质。

七、pygame 桌面版上位机

Web 版（第六章）用浏览器看，适合远程；**pygame 版是原生窗口程序**，适合树莓派接了 HDMI 显示器、或者用 VNC 远程桌面的场合。两者共用同一套串口链路及协议解析，功能一致，区别只是显示层：

	Web 版 (radarpi serve)	pygame 版 (radarpi view)
显示位置	局域网内任意手机/电脑浏览器	树莓派本机屏幕 (HDMI / VNC)
依赖	无 (纯标准库)	python3-pygame、python3-numpy
额外能力	多客户端同时看、手机可看	更低延迟、可离网运行、 原生 ndarray 矩阵视图
无人值守	推荐 (systemd 常驻)	需要有人开界面，或配合 --exit-after 做定时抓取

7.1 安装依赖

```
sudo apt install -y python3-pygame python3-numpy fonts-noto-cjk
```

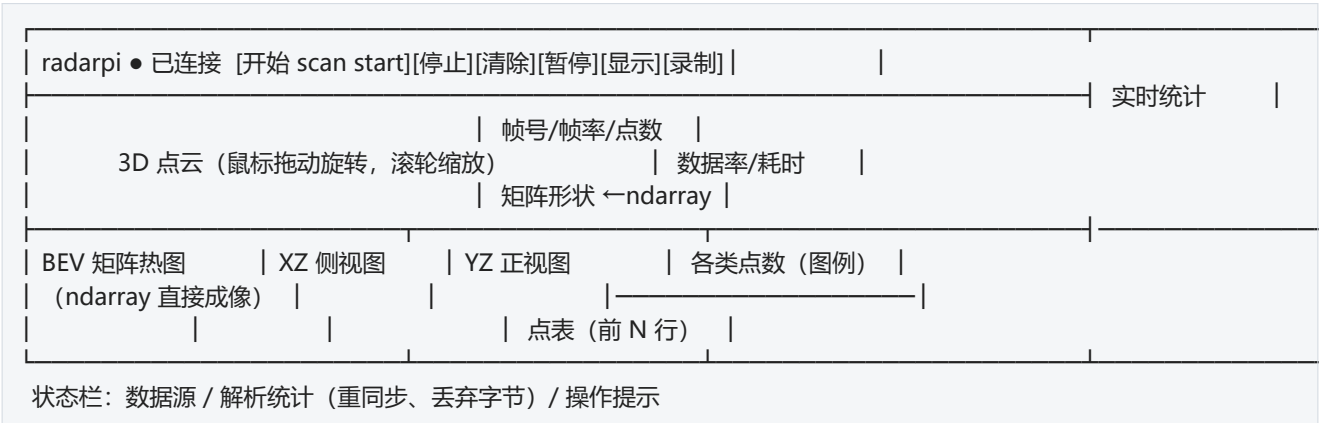
- fonts-noto-cjk 提供中文字体。**没装的话界面会自动切换成英文**（不会出现方块乱码），所以这不是必须项，但装了体验更好：sudo apt install fonts-noto-cjk。
- 如果只跑 Web 版，上面这些都可以不装。

7.2 启动

```
radarpi view          # 接真机
radarpi view --simulate      # 无雷达时先熟悉界面
radarpi view --replay out.jsonl --loop
radarpi view --fullscreen    # 全屏（接显示器做演示时好用）
radarpi view --width 1600 --height 900
```

装了桌面环境的系统，也可以在**应用菜单**里找到"radarpi 雷达上位机"（安装包已写入 /usr/share/applications/radarpi-viewer.desktop）。

7.3 界面说明



左下角的 **BEV 矩阵热图**是 ndarray 接口的可视化：它显示的就是 ndarray_iface.to_bev() 生成的二维矩阵（默认 0.1 m/格），亮的地方点密集。切换 --bev-mode 可以改成显示每格的最大 SNR、最大速度等。右上角的"矩阵形状"显示当前帧 (N, 6) 的 N，用它确认数据确实转换成了矩阵。

界面实拍图见源码目录 docs/pygame_console.png（普通模式）与 docs/pygame_console_processor.png（挂了算法处理器、只显示过滤后的点）。

7.4 快捷键

按键	作用	按键	作用
空格	暂停/继续画面刷新	S	发送 scan start
X	发送 scan stop	C	清除画面
R	开始/停止录制	N	保存矩阵 (.npz)
1~4	全部点/仅动态/仅微动/仅航迹	P	截图 (PNG)
F	全屏切换	Q / ESC	退出
鼠标拖动	旋转 3D 视角	滚轮	缩放

7.5 常用参数

参数	说明				
--bev-resolution 0.05	BEV 矩阵分辨率 (m/格)，越小越精细、越吃 CPU				
--bev-mode count\	max_snr\	max_abs_v\	mean_v\	min_dist、	BEV 每格填什么值
--max-points 2000	每帧最多绘制多少点，画面卡就调小				
--processor ./my_algo.py:process	挂一个算法处理器（见第十五章），实时处理并显示结果				
--buffer 60	"存矩阵"时保存最近多少帧				
--record-format jsonl	录制格式				
--record-dir /var/lib/radarpi	录制与存矩阵的目录				
--snapshot out.png --snapshot-frames 60	画够 60 帧后截图并退出（无显示器自检、定时抓图）				
--exit-after 30	30 秒后自动退出（配合 cron 定时抓图）				

7.6 录制与"存矩阵"

- **录制**（R 键或工具栏）：把原始点云写成 CSV / JSONL，与 Web 版完全相同，详见第九章。
- **存矩阵**（N 键）：把缓冲区里最近 N 帧的 **ndarray 矩阵**存成一个 .npz 文件，供后续离线训练/分析：

```
import numpy as np
d = np.load("points_20260916_171218.npz")
print(d.files)          # ['fields','points_0000',..., 'frame_ids','timestamps','counts']
pts = d["points_0000"]  # (N, 6) float32: x, y, z, snr, v, group
print(d["fields"])      # 列名（防止自己记错列序）
```

无显示器也能用：树莓派跑命令行的场景下，用 `--snapshot` 定时抓图，

```
# 每分钟把当前点云画面存一张 PNG
radarpi view --snapshot /var/lib/radarpi/shot_$(date +%H%M).png --snapshot-frames 90 --exit-after 20
```

7.7 与 Web 版如何选择

- **长期监视、远程查看、手机上看** → 用 Web 版（`radarpi serve`，开机自启）。
- **本机看、要更低延迟、要把算法结果直接画出来** → 用 pygame 版（`radarpi view`）。
- 两者**不要同时**运行：串口是独占的，同一时刻只能有一个程序打开雷达端口。

需要停掉 Web 服务再开 pygame 版：

```
sudo systemctl stop radarpi-web  # 让出串口
radarpi view
sudo systemctl start radarpi-web  # 用完再交回去
```

八、雷达参数配置

配置有四条路径，效果相同，按习惯选一条即可。

7.1 网页下发（最直观）

网页"设置 → 雷达指令"，填好数值点"下发"，等效于在 Windows 上位机里改 Radar Set 面板。指令与手册完全一致：

指令名	手册原文	示例	说明
radar_height	set radar_height 1.7	1.7	雷达离地高度（m）
radar_inclination	set radar_inclination 30	30	安装下倾角（度）
boundary	set boundary -3 3 0.8 0.2 5	-3 3 0.8 0.2 5	点云输出范围，5 个数值；设成与房间边界一致可滤掉房间外杂点
mmsinterval	set_mmsinterval 2 34	2 34	长时/短时微动点处理间隔（帧）
cfar_coeff	set_cfar_coeff 33 44 55	33 44 55	动态/短时微动/长时微动检测门限系数
scan_start	scan start	无	开始发送点云
scan_stop	scan stop	无	停止发送点云

注意手册里指令名写法不统一：radar_height、radar_inclination、boundary 用 **空格**，而 mmsinterval、cfar_coeff 用**下划线**。radarpi 严格按手册原文发送，未做"纠正"，以免固件不识别。指令均以 `\r\n` 结尾。

7.2 命令行下发

```
radarpi send "scan start"
radarpi send "set radar_height 1.7"
radarpi send --key boundary 3 3 1 1 5    # 参数分开写
radarpi send "scan stop; scan start"    # 分号分隔可一次发多条
```

7.3 配置文件（推荐用于固定安装）

编辑 `/etc/radarpi/radarpi.conf`:

```
[radar]
apply_on_connect = true    # 连上雷达后自动下发下面这些参数
radar_height = 1.7
radar_inclination = 30
boundary = -3 3 0.8 0.2 5
mmsinterval = 2 34
cfar_coeff = 33 44 55

[serial]
device = auto              # 或 /dev/radar、/dev/serial/by-id/xxx
baudrate = 3000000
auto_start = true          # 连上后自动 scan start
reconnect = true           # 掉线自动重连

[web]
host = 0.0.0.0
port = 8080
max_points_sent = 4000     # 每帧推送给浏览器的点数上限
max_fps = 15               # 网页刷新上限

[record]
directory = /var/lib/radarpi
format = csv
autostart = false
```

改完执行 `sudo systemctl restart radarpi-web` 生效。也可以在网页"设置 → 配置文件"里改并点"保存到配置文件"（串口/端口等条目仍需重启服务）。

7.4 命令行改配置

```
radarpi config list
radarpi config get radar.boundary
radarpi config set radar.boundary "-3 3 0.8 0.2 5"
radarpi apply          # 立刻把配置里的参数下发到雷达（不用重启服务）
```

九、数据录制与回放

8.1 录制

网页点"开始录制", 或命令行:

```
radarpi record /var/lib/radarpi/room1.csv    # CSV, 默认格式
radarpi record /var/lib/radarpi/room1.jsonl -f jsonl # JSONL
radarpi record out.csv --duration 60          # 录 60 秒自动停
```

CSV (一行一个点, 便于 Excel / pandas 分析)

```
timestamp,unix_time,frame_id,group,group_name,x_m,y_m,z_m,snr,speed_mps
2026-09-16T17:12:18.475,1789549898.475,172,1,动态低置信度点,-0.597,5.013,0.676,306,0.00
```

group: 0=动态高, 1=动态低, 2=长时微动高, 3=长时微动低, 4=短时微动高,
5=短时微动低, -1=航迹。

JSONL (一行一帧, 含帧头信息, 回放最快, 无损)

```
{"ts":1789549898.475,"header":{"frame_id":172,"frame_period":100,...,"counts":{"...}},
"points":[{"x":-0.597,"y":5.013,"z":0.676,"snr":306,"v":0.0,"g":1},...]}
```

文件超过 256 MB 会自动分卷 (radar1_001.csv、radar1_002.csv...)。

8.2 回放

```
radarpi replay /var/lib/radarpi/room1.jsonl      # 终端看统计
radarpi replay /var/lib/radarpi/room1.jsonl --loop --speed 2
radarpi replay /var/lib/radarpi/room1.csv --serve # 在网页里回放
radarpi serve --replay "/var/lib/radarpi/room1*.jsonl" --loop # 网页多文件巡回回放
```

回放走的是和真机完全相同的显示与统计管线, 因此**可以用来复现问题、做演示、或者在没有雷达的现场培训。**

十、命令行参考

```
radarpi doctor [--probe] [--seconds N] [--json]  环境体检 / 真机联调
radarpi ports [--json]                          列出串口设备
radarpi serve [--host H] [--port P] [--simulate] [--replay FILE] [--loop]
                                                    启动 Web 上位机
radarpi view [--fullscreen] [--bev-mode M] [--processor MOD:FUNC]
                                                    启动 pygame 桌面上位机
radarpi monitor [--json|--raw] [--duration N]     终端看实时数据
radarpi record FILE [-f csv|jsonl] [--duration N] 录制
radarpi replay FILE [--loop] [--speed X] [--serve] 回放
radarpi send "指令" | send --key KEY v1 v2 ...   下发指令
radarpi apply                                     下发配置里的雷达参数
radarpi config list|get|set|path                 配置管理
radarpi sniff [--hex] [--seconds N]             原始抓包 (核对协议)
radarpi version                                  版本
```

通用参数: --device (串口号/auto//dev/radar)、--baud (默认 3000000)、
--simulate (用仿真数据)、--replay FILE (回放文件)、--config (指定配置文件)。

radarpi serve --record-to FILE 可以在启动网页的同时开始录制 (适合做无人值守记录)。

radarpi view 的显示与算法相关参数 (详见第七章): --bev-resolution、--bev-mode、
--max-points、--buffer、--processor 模块函数、--snapshot PNG --snapshot-frames N、--exit-after 秒。

十一、开机自启与无人值守

安装后 radarpi-web 服务已经开机自启，串口掉线会自动重连（reconnect = true），重连成功后会自动重新下发配置并恢复 scan start，所以现场断电恢复后无需人工干预。

```
systemctl status radarpi-web    # 状态
sudo systemctl restart radarpi-web # 重启（改过配置后）
sudo systemctl stop radarpi-web  # 临时停（会把串口让给别的程序）
journalctl -u radarpi-web -f     # 实时日志
journalctl -u radarpi-web -n 200 # 最近 200 行
sudo systemctl disable radarpi-web # 取消开机自启
```

无人值守长期录制示例（每天一个文件）：

```
# 在 /etc/radarpi/radarpi.conf 里设
# [record] autostart = true ; format = jsonl
# 再用 cron 每天重启一次服务，即可实现"一天一个文件"
sudo crontab -e
0 0 * * * /bin/systemctl restart radarpi-web
```

磁盘空间：CSV 大约每 1000 点 60 字节，按 10 fps、每帧 300 点估算约 18 MB/小时。建议定期清理或用 df -h 观察。

十二、故障排除

现象	可能原因	处理方法	
radarpi ports 看不到设备	线没插好 / 只插了 VCC 没插 GND / 小板没供电	换 USB 口、换线；lsusb 看芯片是否出现	
有设备节点但打不开（Permission denied）	用户不在 dialout 组	sudo usermod -aG dialout \$USER 后 重新登录 ；或先用 sudo radarpi serve	
有 ttyUSB0 但报"资源忙/设备忙"	brltty 或 ModemManager 抢占了串口	sudo apt remove brltty；安装包已写入屏蔽规则，若仍不行重启一次	
能打开串口但 一帧都解不出来	波特率不对 / 小板不支持 3 Mbit/s / 线太长	radarpi sniff --hex 看有无数据；换支持 3000000 的 FT232 小板；缩短 USB 线	
帧率低、频繁"重同步"	串口误码、USB 带宽不足、树莓派欠压	换短且质量好的线；确认电源 5V/3A；vcgencmd get_throttled 应为 0	
能看到点但目标很稀疏	门限系数过大 / 边界设太窄	调 cfar_coeff；把 boundary 放大到覆盖整个房间	
房间里满屏杂点	边界未与房间对齐、门限太小	用 set boundary 把输出范围设成与房间一致；适度提高 cfar_coeff	
网页打不开	服务没起 / 端口占用 / 防火墙 / 不在同一网段	systemctl status radarpi-web；`ss -ltnp \	grep 8080；hostname -I 确认 IP
网页卡顿、掉帧	推送点数/帧率过高，或 Wi-Fi 弱	调低 max_points_sent（如 1500）与 max_fps（如 8）；改用网线	
树莓派重启、日志报欠压	供电不足（雷达从树莓派取电）	雷达改独立电源；换官方 5V/3A 适配器	

现象	可能原因	处理方法	
页面状态灯黄色、长时间无帧	雷达未 scan start / 雷达已停机	网页点"开始 scan start"; 检查雷达通信灯是否闪烁	
radarpi view 报"pygame 未安装"	缺依赖	sudo apt install -y python3-pygame python3-numpy	
pygame 窗口里中文显示成方块/英文	系统没有中文字体 (界面会自动切英文)	sudo apt install fonts-noto-cjk (装完重启程序)	
radarpi view 报串口被占用	后台的 Web 服务正在用同一个串口	sudo systemctl stop radarpi-web, 用完再 start	
pygame 画面卡顿	点数太多、窗口太大、CPU 降频	调小 --max-points 1500、降低分辨率、vcgencmd measure_temp 查温度	
VNC/SSH 里跑 radarpi view 报 "no available video device"	没有 X 显示 (纯命令行)	用 Web 版; 或 export DISPLAY=:0 (有桌面会话时); 或 --snapshot 抓图	
需要存矩阵做算法但不想开界面	—	用 radarpi view --snapshot 抓图, 或直接写脚本调 ndarray_iface (第十五章)	

进一步排查:

```
radarpi doctor --probe      # 一键体检 + 联调
radarpi sniff --hex --seconds 5 # 看原始报文 (核对包头 FFEFFDC / FFDDFECB)
dmesg | tail -30           # 看内核有没有报 USB/串口错误
journalctl -u radarpi-web -n 100
```

十三、日常维护

- **每周**: 检查接线是否松动, radarpi doctor 看链路统计 (resyncs 是否在涨)。
- **每月**: 用柔软干布擦拭雷达天线罩, 避免化学清洁剂; 清理 /var/lib/radarpi 里的旧录制文件。
- **每季度**: 检查防水、接地; sudo apt update && sudo apt full-upgrade 更新系统 (更新内核后重启一次)。
- **每年**: 按原手册做一次性性能校准 (需要 Windows 上位机)。
- 建议把 radarpi 源码与录制文件定期备份, 配置备份 /etc/radarpi/radarpi.conf 一个文件即可。

十四、技术指标 (雷达)

摘自原手册, 供选型与验收参考:

项目	参数
工作频率	60 GHz
水平 / 垂直探测角度	-40°~+40° / -30°~+30°

项目	参数
距离 / 距离分辨率	20 m / ≤ 0.3 m (@10 m)
水平角度分辨率	$\leq 1^\circ$
速度测量范围	-50 ~ +50 m/s
工作温度 / 防护等级	-40°C ~ +85°C / IP67
电源 / 功耗	5-12V DC / ≤ 3 W
安装高度 / 下倾角	1.5~2.5 m / $0^\circ \sim 20^\circ$
串口	3000000 bit/s, 8 数据位, 1 停止位, 无校验

十五、ndarray 矩阵接口（算法接入）

这一章是给“拿到数据之后要做算法”的人看的：上位机（Web / pygame）只是这套接口的调用方之一，你完全可以不用界面，把雷达接到自己的脚本、模型或 ROS 节点里。

15.1 数据契约（最重要的一条）

所有矩阵接口都用同一个点云矩阵格式：

```
points : np.ndarray, shape = (N, 6), dtype = float32
```

列号	名称	单位 / 取值	说明
0	x	m	横向，右为正
1	y	m	前向距离，雷达正前方为正
2	z	m	高度，地面为 0
3	snr	线性值	只反映相对强弱， 不是绝对值
4	v	m/s	径向速度，靠近为负
5	group	0~5	点类别；-1 表示航迹

类别编号：0 动态高置信度、1 动态低置信度、2 长时微动高置信度、3 长时微动低置信度、4 短时微动高置信度、5 短时微动低置信度。

代码里请用现成常量而不是手写数字：nd.GROUP_DYNAMIC、nd.GROUP_DYNAMIC_HIGH、nd.GROUP_MICRO、nd.POINT_FIELDS。

15.2 最小示例

```
from radarpi.link import RadarLink
from radarpi import ndarray_iface as nd

link = RadarLink(device="auto")          # 真机
# 也可用 SimulatedLink() (仿真) 或 ReplaySource("out.jsonl") (回放)
for points, frame in nd.stream_arrays(link, max_frames=100):
    print(points.shape, frame.frame_id)   # (262, 6) 15
    dyn = nd.filter_points(points, groups=nd.GROUP_DYNAMIC) # 只留动态点
    bev = nd.to_bev(dyn, resolution=0.05)      # (H, W) 俯视图矩阵
    vox = nd.to_voxel(dyn, resolution=0.2)      # (NX, NY, NZ) 体素
```

stream_arrays() 会自动在退出时关闭数据源。

15.3 帧 → 矩阵

函数	说明
<code>frame_to_ndarray(frame)</code>	帧 → (N, 6) 矩阵 (最常用)
<code>tracks_to_ndarray(frame)</code>	航迹 → (M, 6) 矩阵 (最后一列是航迹号)
<code>frame_to_bundle(frame)</code>	打包成字典 (points/tracks/counts/frame_id/timestamp/frame_period), 可直接 <code>np.savez_compressed("f.npz", **bundle)</code>
<code>ndarray_to_points(matrix)</code>	矩阵 → Point 列表 (把算法结果交回界面渲染时用)

15.4 栅格化: BEV / 体素矩阵

`to_bev(points, resolution=0.1, x_range=(-5,5), y_range=(0,10), mode="count", groups=None)`

返回 (H, W) float32。坐标约定 (务必记牢):

- 行对应 Y (前向), 行 0 是最近处 (图像习惯); 列对应 X (横向), $w//2$ 是 $X=0$;
- 世界坐标 $\square$ 行列换算用 `bev_extent()` 与 `bev_to_world()`。

`mode` 决定每格填什么:

mode	含义	典型用途
count	点数 (默认)	占据栅格、建图、CNN 输入通道
max_snr	该格最大 SNR	找强反射目标
max_abs_v	最大速度绝对值	找运动目标
mean_v	平均速度 (带符号)	估计运动方向
min_dist	最近距离	障碍物距离图

`to_bev_channels(points, channels="groups")` 直接给多通道张量:

- `channels="groups"` → (6, H, W), 6 类点各一个通道;
- `channels="fields"` → (3, H, W), 依次是点数、最大 SNR、最大速度绝对值。

需要 (H, W, C) 时自己 `transpose(1, 2, 0)`。

`to_voxel(points, resolution=0.1, z_range=(0,3))` → (NX, NY, NZ) uint16, 与 X/Y/Z 同序, 可直接接 3D 卷积。

15.5 过滤

`filter_points(points, groups=..., x_range=..., y_range=..., z_range=..., min_snr=..., min_speed=..., max_speed=...)`

返回同格式矩阵, 可链式使用。常用组合:

```
# 房间中间、地面以上、运动中的目标 (滤掉墙面地面与静态杂点)
target = nd.filter_points(points, groups=nd.GROUP_DYNAMIC,
                          x_range=(-3, 3), y_range=(1, 8), z_range=(0.25, 2.2))
```

15.6 处理器插件: 把算法挂到界面上

界面的 `--processor` 参数可以挂一个外部 Python 函数, 每帧调用一次:

```
radarpi view --processor ./examples/processor_demo.py:process
radarpi view --processor my_package.detect:process # 也可以按模块名加载
```

函数约定: `fn(points, frame) -> ndarray | None`

- 返回 **新的 (N, 6) 矩阵** → 界面显示的就是处理后的点云 (可以用来验证算法效果) ;
- 返回 `None` → 不修改显示, 只做观测/记录/喂模型。

仓库里的 `examples/processor_demo.py` 给了三个可直接抄的样板: `process` (区域+类别过滤)、`observe_only` (只统计 BEV/体素的形状与峰值)、`cluster_by_grid` (网格粗聚类, 换成 DBSCAN 就是真实聚类) 。

pygame 界面右上角会显示处理器文件名与每帧耗时, 右侧"矩阵形状"会跟着变成处理后的 N ——这两处是判断插件是否生效最快的办法。

15.7 性能参考 (树莓派 4B)

操作	量级
<code>frame_to_ndarray</code> (约 300 点)	< 0.1 ms
<code>to_bev</code> 0.1 m/格、视野 10m×10m	约 0.1~0.3 ms
<code>to_bev_channels</code> (6 通道)	约 1 ms
<code>to_voxel</code> 0.1 m/格	约 0.5 ms
示例处理器 (区域过滤)	0.1~0.2 ms

数据量本身很小 (每帧几十到几千点) , 瓶颈通常在显示与网络, 不在矩阵转换。

15.8 下游算法接口 (伪代码)

```
# 1) 点云聚类 + 跟踪
for points, frame in nd.stream_arrays(link):
    target = nd.filter_points(points, groups=nd.GROUP_DYNAMIC, z_range=(0.2, 2.0))
    labels = my_dbscan(target[:, :3], eps=0.3, min_samples=5) # 自己的聚类
    for cid in set(labels) - {-1}:
        center = target[labels == cid, :3].mean(axis=0)
        tracker.update(center)

# 2) 占用栅格 / 建图
bev = nd.to_bev(points, resolution=0.05, mode="count")
occupancy = (bev > 0).astype(np.uint8)

# 3) 神经网络输入 (时序 + 多通道)
stack = np.stack([nd.to_bev_channels(p, resolution=0.1) for p in last_10_frames])
# stack.shape == (10, 6, 100, 100) → 送 3D/时序网络

# 4) 数据集制作: 界面里按 N 存 .npz, 或脚本里自己攒
buffer = []
for points, frame in nd.stream_arrays(ReplaySource("room1.jsonl")):
    buffer.append(points) # 每帧形状可变, 用 list 存
np.savez_compressed("dataset.npz", **{"points_%04d" % i: a for i, a in enumerate(buffer)})
```

15.9 不接硬件也能开发

```
from radarpi.link import SimulatedLink
sim = SimulatedLink(fps=10.0)      # 6m×5m 房间 + 走动的人
from radarpi import ndarray_iface as nd
for points, frame in nd.stream_arrays(sim, max_frames=5):
    ...
```

或者回放已有录制（`ReplaySource("out.csv", loop=True, speed=2.0)`），接口与真机完全一致，因此算法可以先在没有雷达的电脑上写完、再上树莓派。

附录 A：CH343/CH344 串口小板驱动

Linux 主线内核**没有** ch343 驱动（内核里的 `ch341.c` 只覆盖 CH340/CH341，USB ID 不含 CH343 的 `1a86:55d3`）。实测路径有两条，先试第一条：

路径 1：什么都不装，先插上看看

```
lsusb          # 能看到 1a86:55d3 等
ls /dev/ttyACM* /dev/ttyUSB* # 出现 /dev/ttyACM0 就说明 CDC-ACM 路径可用
radarpi doctor --probe
```

多数 CH343/CH344/CH9102 模块本身兼容 CDC-ACM，会被内核 `cdc_acm` 直接枚举成 `/dev/ttyACM0`，**无需安装任何驱动**即可使用。

路径 2：路径 1 失败时，安装 WCH 厂商驱动

```
sudo ./packaging/install-ch343-driver.sh --check # 先看是否需要
sudo ./packaging/install-ch343-driver.sh          # 联网编译安装（DKMS）
```

脚本会安装内核头文件、拉取 WCH 官方驱动源码、自动修正内核 6.12+ 的 `linux/unaligned.h` 头文件改名问题，并以 DKMS 方式安装（内核升级后自动重编译）。装好后设备名为 `/dev/ttyCH343USB0`。

如果只是想省事：**换一块 FT232 串口小板**。Linux 内核自带 `ftdi_sio`，实测支持到 3000000 波特率（FT232R 的上限正好是 3 Mbaud），插上即用、升级内核也不受影响。

附录 B：数据协议说明

手册第四节给出的协议（radarpi 按此实现）：

B.1 帧头包（28 字节）

字段	位宽	说明
包头	32	固定 0xFFEEFFDC
帧周期	10	32bit 字的 bit0~9
动态高置信度点数	10	bit10~19
Reserved	12	bit20~31
帧号	32	—
动态低置信度点数	10	下一字 bit0~9
跟踪航迹数	10	bit10~19
长时微动高置信度点数	10	bit20~29
Reserved	2	bit30~31

字段	位宽	说明
长时微动低置信度点数	10	下一字 bit0~9
短时微动高置信度点数	10	bit10~19
短时微动低置信度点数	10	bit20~29
Reserved	2	bit30~31
上帧 BB 处理时间	8	uint8, ms
上帧 Post-BB 处理时间	8	uint8, ms
上帧结果传输时间	8	uint8, ms
上帧帧间隔	8	uint8, ms
包尾	32	固定 0xFFEEFFD3

B.2 点云包

字段	位宽	说明
点云包头	32	固定 0xFFDDFECB
点 1 X / Y / Z	16×3	单位 0.001 m, int16
点 1 SNR	16	线性值, uint16 (只反映相对大小, 非绝对值)
点 1 速度	16	单位 0.1 m/s, int16
... (点数由帧头给出)		
Reserved	32	—
包尾	32	固定 0xFFDDFEC4

点的排列顺序：动态高置信度 → 动态低置信度 → 长时微动高 → 长时微动低 → 短时微动高 → 短时微动低。

B.3 航迹包

包头固定 0xFFCCFDBA，随后每条航迹 X/Y/Z (0.001 m, int16) 。

手册在此处截断，未列出航迹是否还包含 SNR 与速度字段。radarpi 的解析器会自动判别 6 字节（仅 X/Y/Z）与 10 字节（含 SNR/速度）两种布局，接真机后可用 radarpi sniff 确认一次；若解析统计里 bad_lengths 持续增长，请把实测结果 反馈给我们以固定该参数。

B.4 两处主动容错（与"照抄手册"的不同之处）

- **字节序自动判定**：手册未说明多字节字段的字节序。解析器在第一次命中包头时自动判定大端/小端，避免猜错导致完全收不到数据。
- **自同步**：半帧接入、串口丢字节、噪声之后，解析器会自动丢弃无效字节并从下一个包头恢复，不会因为一次误码就永久卡死；恢复次数记在 resyncs / dropped_bytes 统计里，网页状态栏与 radarpi monitor 都能看到。

附录 C：与原 Windows 上位机的差异

功能	Windows 上位机	radarpi (树莓派)
运行环境	Windows + Qt 界面	Raspberry Pi OS；界面可选浏览器或 pygame 原生窗口
串口设置	手动选 COM 口	自动识别，支持 /dev/radar 别名

功能	Windows 上位机	radarpi (树莓派)
点云显示	3D + 三投影 + 点表	一致 (Web 版手机也能看; pygame 版另有 BEV 矩阵热图)
参数配置	Radar Set 面板	网页"雷达指令" / 配置文件 / 命令行
数据保存	Data Path 指定目录	CSV / JSONL (自动分卷) + .npz 矩阵包
回放	Playback	radarpi replay / radarpi view --replay, 支持倍速/循环
算法二次开发	无开放接口	ndarray 矩阵接口 + --processor 插件 (第十五章)
固件升级 Upgrade	□	□ 未移植 (协议未公开, 误操作风险高)
校准向导	□	□ 未移植 (需厂商协议)
无人值守	需保持电脑开机	systemd 常驻, 开机自启、断线自愈
远程查看	需远程桌面	局域网浏览器直接访问

建议的混合用法：日常监视、数据采集、参数微调全部用树莓派；固件升级与年度校准 再拿一台 Windows 电脑接上原上位机即可，两者互不冲突（同一时间只能有一个程序占用 串口）。